Agentic Core

> **v3.3.1** — Framework profissional para orquestração de Agentes Autônomos em Node.js com suporte a múltiplos provedores redundantes (Google Gemini, OpenAI, Claude, Ollama, Nvidia), **ferramentas nativas de raciocínio passo a passo (`reasoningTool`)**, **cálculo determinístico em sandbox segura (`calculatorTool`)**, **pesquisa web gratuita e sem API keys (`webSearchTool`)**, **streaming em tempo real com `TokenStream`**, **auto-RAG de memória com `ContextAugmenter`**, **segurança avançada com `ToolSecurityPolicy`, anti-SSRF e path confinement**, **orquestração em DAG condicional**, observabilidade e telemetria com `TelemetryProvider`, suporte nativo ao protocolo MCP (Model Context Protocol), failover automático em falhas 5xx, suporte a mídias multimodais e concorrência transparente.

✨ Características

Recurso	Descrição
Raciocínio Passo a Passo (v3.3.1)	`reasoningTool: true` ativa a ferramenta de *Sequential Thinking* (`record_thought_step`). O agente decompõe problemas complexos em etapas, hipóteses e revisões com emissão de eventos `AgentEvents.REASONING_CHUNK`.
Cálculo Determinístico PAL (v3.3.1)	`calculatorTool: true` (ou `mathTool`) disponibiliza a ferramenta `calculate_expression`. Avalia expressões matemáticas em sandbox isolada `node:vm` protegida contra RCE com timeout de 1.500ms e precisão exata.
Pesquisa Web Nativa e Gratuita (v3.3.1)	`webSearchTool: true` (ou `searchTool`) fornece a ferramenta `web_search`. Realiza buscas em tempo real na internet via DuckDuckGo sem chaves de API, com extração limpa de snippets e proteção anti-SSRF.
Compatibilidade com SLMs (v3.3.1)	Roteamento inteligente de intenção e parser resiliente de chamadas XML/JSON, garantindo **100% de precisão de acerto** comprovado em modelos ultracompactos locais como **`qwen3:0.6b`**.
Streaming & Realtime (v3.3)	`processMessageStream()` e `TokenStream` com suporte a `for await (const chunk of stream)` e eventos para tokens parciais, raciocínio (`thought`), chamadas e retornos de ferramentas.
Auto-RAG de Memória (v3.3)	`ContextAugmenter` para recuperação e injeção automática de fatos duradouros e episódios passados no contexto do agente (`autoRetrieveMemory: true`).
Segurança & Sandboxing (v3.3)	`ToolSecurityPolicy` com rate limiting por sessão, listas de bloqueio/permissão de ferramentas, proteção anti-SSRF em `WebTools` e confinamento de caminho contra path traversal em `FileTools`.
DAG Condicional & Reativo (v3.3)	`TaskGraph` e `ParallelExecutor` com suporte a branches condicionais (`condition`), transformação de contexto entre nós (`transformContext`) e tarefas via funções diretas (`fn`).
Telemetria & Observabilidade (v3.3)	`TelemetryProvider` com rastreamento hierárquico de spans (`startSpan`/`endSpan`), latência (`durationMs`), status e contabilidade acumulada de tokens (`prompt`, `completion`, `total`).
Multi-Provider Nativo	Suporte a **Google Gemini**, **OpenAI GPT**, **Anthropic Claude**, **Ollama** (modelos locais) e **Nvidia NIM**.
Redundância e Failover	Múltiplos modelos/provedores com `currentIndex` atômico. Transição imediata em falhas 5xx, rate limits ou timeouts.

Suporte Nativo a MCP	Integração completa com o protocolo MCP (Model Context Protocol), atuando tanto como Cliente MCP quanto Servidor MCP via Stdio/SSE.
Memória Hierárquica	`WorkingMemory` (janela deslizante), `ContextCompactor` (resumos via LLM), `EpisodicMemory` (episódios significativos) e `SemanticMemory` (fatos e preferências).
SubAgentes Isolados	`SubAgent` para execução de tarefas especializadas em segundo plano, com gestão efêmera de sessão e desregistro automático de ferramentas temporárias.
Plataforma Multi-Tenant	`PlatformManager` para gestão de múltiplos tenants, agentes e sessões isoladas.
Persistência de Sessões	`SessionStore` e `InMemorySessionStore` para salvamento, restauração e exclusão persistente de estado.
Suporte Multimídia	Envio de anexos (imagens, áudio, vídeo, PDFs) em Base64 no processamento de mensagens.
Concorrência Transparente	Gerenciamento automático de mensagens consecutivas (`debounceMs`) com cancelamento ativo no LLM.
Agentic Loop Completo	Tool calls encadeados com execução recursiva e contextualizada.
Gerenciamento de Sessões	TTL configurável com renovação automática por atividade e normalização transparente de chaves (string/número).
Sistema de Ferramentas	**18+ ferramentas embutidas** organizadas em categorias (reasoning, web, file, database, system, communication) com validação de schema, composição, chaining e marketplace.

📦 Instalação

Via npm (Registro Público Oficial)

```bash
npm install @areumtecnologia/agentic-core
```

Via GitHub Packages

```bash
npm install github:areumtecnologia/agentic-core
```

Via clone local

```bash
git clone https://github.com/areumtecnologia/agentic-core.git
cd agentic-core
npm install
```

Pré-requisitos

- Node.js `>=18.0.0`
- Chave de API de um provedor compatível (Google Gemini, OpenAI, Anthropic, Nvidia) ou Ollama rodando localmente

---

⚛ Configuração

Copie o arquivo de exemplo e configure suas credenciais:

```bash
cp .env.example .env
```

```env
# .env
GOOGLE_GEMINI_API_KEY=sua-chave-aqui
OPENAI_API_KEY=sua-chave-openai-aqui
ANTHROPIC_API_KEY=sua-chave-anthropic-aqui
NVIDIA_API_KEY=sua-chave-nvidia-aqui
```

🚀 Quickstart

Exemplo Básico com Google Gemini

```javascript
require('dotenv').config();
const {
  AgenticCore,
  AgentConfig,
  Type,
  GoogleProvider
} = require('@areumtecnologia/agentic-core');

// 1. Configurar o agente (suporta construtor direto ou encadeamento fluente)
const agentConfig = new AgentConfig()
  .setName('Monnalisa')
  .setCompany('Áreum Tecnologia', 'Soluções em IA e Automação de Processos.')
  .setMission('Agente de Vendas', 'Atuar como assistente virtual e agente de
vendas.', 'Atenda os clientes cordialmente.')
  .setReasoningLanguage('pt-BR');

// 2. Instanciar o agente com provedor
const agent = new AgenticCore({
  provider: new GoogleProvider({
    apiKey: process.env.GOOGLE_GEMINI_API_KEY,
    model: 'gemma-4-26b-a4b-it'
  }),
  debounceMs: 1500,
  agent: agentConfig,
});

// 3. Registrar ferramentas
agent.registerTool({
```

```
  name: 'get_product_info',
  description: 'Obtém informações de produtos disponíveis.',
  parameters: {
    type: Type.OBJECT,
    properties: {
      category: { type: Type.STRING, description: 'Categoria do produto.' },
    },
  },
}, async ({ category }, signal) => {
  return JSON.stringify({ products: ['Produto A', 'Produto B'] });
});

// 4. Criar sessão (aceita string ou número como ID)
const session = agent.createSession('session-001', {
  name: 'João Silva',
  phone: '+55 11 98765-4321',
  email: 'joao@exemplo.com',
});

// 5. Processar mensagens
const response = await agent.processMessage(session.id, 'Olá!');
console.log(response.response);
```

🛠 Ferramentas Nativas Embutidas (v3.3.1)

A versão 3.3.1 adiciona **três ferramentas nativas de primeira classe** que podem ser ativadas diretamente via flags no construtor do `AgenticCore`, sem necessidade de configuração manual ou chaves de API externas:

```javascript
const agent = new AgenticCore({
  agent: agentConfig,
  provider: new OllamaProvider({ model: 'qwen3:0.6b' }),
  calculatorTool: true, // Avaliação matemática determinística isolada
  reasoningTool: true,  // Pensamento sequencial estruturado (scratchpad)
  webSearchTool: true   // Pesquisa na Web gratuita em tempo real
});
```

1. 🧮 Cálculo Matemático Determinístico (`calculatorTool`)

Modelos de linguagem frequentemente alucinam em operações aritméticas ou cálculos encadeados. Com `calculatorTool: true` (ou `mathTool: true`), o agente recebe a ferramenta `calculate_expression`, delegando cálculos para um motor determinístico seguro baseado em `node:vm`:

- **Sandbox Blindada**: Escopo isolado sem acesso a `process`, `require`, `fs`, `global` ou protótipos de execução (proteção anti-RCE).
- **Timeout Estrito**: Interrupção forçada em no máximo 1.500ms contra loops infinitos.
- **Funções Matemáticas Suportadas**: Todas as funções do objeto padrão `Math`

(`Math.sqrt`, `Math.pow`, `Math.sin`, `Math.round`, etc.) e operadores padrão (`+`, `-`, `*`, `/`, `%`, `**`).

```javascript
// O agente chama automaticamente calculate_expression para responder:
const res = await agent.processMessage(session.id, 'Quanto é (45 * 12) + (130 / 2) - 15?');
console.log(res.response); // "O resultado é 590."
```

2. 🧩 Raciocínio Sequencial Passo a Passo (`reasoningTool`)

Inspirado no paradigma *Sequential Thinking*, a flag `reasoningTool: true` ativa a ferramenta `record_thought_step` (alias `recordThoughtStep`). O agente utiliza um scratchpad dinâmico para estruturar problemas complexos antes de sintetizar a resposta final:

- **Metadados Estruturados**: Cada passo registra `thoughtNumber`, `totalThoughts`, `isRevision`, `revisesThought` e `hypothesis`.
- **Emissão em Tempo Real**: Cada etapa de pensamento emite o evento `AgentEvents.REASONING_CHUNK` no agente e eventos do tipo `thought` no streaming.
- **Histórico Consolidado**: O raciocínio acumulado é retornado em `result.reasoning` e salvo no histórico da sessão.

```javascript
agent.on(AgentEvents.REASONING_CHUNK, ({ thought, thoughtNumber, totalThoughts }) => {
  console.log(`[Pensamento ${thoughtNumber}/${totalThoughts}]: ${thought}`);
});

const res = await agent.processMessage(
  session.id,
  'Analise os prós e contras de migrar uma arquitetura monolítica para microsserviços passo a passo.'
);
```

3. 🌐 Pesquisa na Web Nativa e Gratuita (`webSearchTool`)

Com `webSearchTool: true` (ou `searchTool: true`), o agente ganha a capacidade de consultar a internet em tempo real através da ferramenta `web_search`:

- **100% Gratuita e sem API Keys**: Utiliza o motor de busca do DuckDuckGo HTML com raspagem semântica robusta.
- **Decodificação de Links**: Resolve automaticamente URLs de redirecionamento interno (`//duckduckgo.com/l/?uddg=...`).
- **Segurança Anti-SSRF Integrada**: Bloqueia endereços IP privados, localhost e metadados de nuvem.

```javascript
const res = await agent.processMessage(
  session.id,
  'Pesquise as novidades mais recentes lançadas no Node.js 22 e resuma os
```

````
principais pontos.'
);
console.log(res.response);
```

### ⚡ 100% de Precisão Comprovada com Modelos Compactos (SLMs)

O `AgenticCore` conta com roteamento inteligente de intenção e parser resiliente de chamadas XML/JSON (`<tool_call>...`), garantindo que até mesmo modelos ultracompactos locais rodem ferramentas com 100% de assertividade:

- **Bateria de Testes com `qwen3:0.6b` (Ollama local)**:
  - 🔢 Cálculo Matemático: **10/10 (100% de sucesso)**
  - 🧩 Raciocínio Sequencial: **10/10 (100% de sucesso)**
  - 🌐 Pesquisa na Web: **10/10 (100% de sucesso)**
  - **Total Geral: 30/30 (100% de aprovação)**

---

## ⚡ Streaming em Tempo Real (v3.3)

O Agentic Core v3.3 introduz suporte nativo a streaming completo de tokens e pensamentos em tempo real através do `TokenStream`:

```javascript
const { AgenticCore, AgentConfig, OpenAIProvider } =
require('@areumtecnologia/agentic-core');

const agent = new AgenticCore({
  agent: new AgentConfig().setName('StreamAgent').setReasoningLanguage('pt-BR'),
  provider: new OpenAIProvider({ apiKey: process.env.OPENAI_API_KEY, model:
'gpt-4o' })
});

const session = agent.createSession('sess-stream-1', { name: 'Cliente' });

// Inicia streaming em tempo real
const stream = agent.processMessageStream(session.id, 'Explique a teoria da
relatividade resumidamente.');

// Consumo via iteração assíncrona
for await (const chunk of stream) {
  if (chunk.type === 'token') {
    process.stdout.write(chunk.text);
  } else if (chunk.type === 'thought') {
    console.log(`\n[Raciocínio interno: ${chunk.thought}]\n`);
  } else if (chunk.type === 'tool_call') {
    console.log(`\n[Executando ferramenta: ${chunk.toolCall.name}]...\n`);
  }
}

// A resposta final estruturada pode ser obtida ao final
const finalResult = await stream.response;
console.log('\nFinalizado:', finalResult.sent_at);
````

```

---

## 🧠 Memória & Auto-RAG (v3.3)

Com `autoRetrieveMemory: true`, o `ContextAugmenter` pesquisa automaticamente memórias semânticas e episódicas antes de cada turno e enriquece a mensagem enviada ao modelo:

```javascript
const { AgenticCore, InMemoryStore, AgentConfig, GoogleProvider } =
require('@areumtecnologia/agentic-core');

const store = new InMemoryStore();
const core = new AgenticCore({
  agent: new AgentConfig().setName('SupportAgent'),
  provider: new GoogleProvider({ apiKey: process.env.GOOGLE_GEMINI_API_KEY }),
  memoryStore: store,
  autoRetrieveMemory: true // Ativa RAG automático de memória
});

// Ensina fatos à memória duradoura do cliente
await core.semanticMemory.learn({
  sessionId: 'sess-001',
  subject: 'plano_usuario',
  predicate: 'tipo',
  object: 'Plano Enterprise VIP'
});

const session = core.createSession('sess-001', { name: 'Carlos' });

// O modelo recebe automaticamente o contexto histórico relevante!
const response = await core.processMessage(session.id, 'Qual é o meu plano?');
console.log(response.response);
```

🛡️ Segurança, Sandboxing e Políticas de Ferramentas (v3.3)

1. `ToolSecurityPolicy` (Rate Limiting e Listas de Bloqueio)

Controle quais ferramentas podem ser chamadas e com que frequência:

```javascript
const { ToolSecurityPolicy, AgenticCore } =
require('@areumtecnologia/agentic-core');

const securityPolicy = new ToolSecurityPolicy({
  blockedTools: ['execute_command', 'delete_database'],
  maxInvocationsPerMinute: 20 // Limite de chamadas por minuto por sessão
});
```
```

```
const core = new AgenticCore({
  agent,
  provider,
  securityPolicy // Aplicado globalmente em todas as execuções de tools
});
```

2. Confinamento de Diretório em `FileTools`

Evita ataques de *path traversal* (como `../../etc/passwd`) e bloqueia leitura de arquivos confidenciais (`.env`, `.git`, chaves privadas, certificados):

```javascript
const { FileTools } = require('@areumtecnologia/agentic-core');

const fileTools = new FileTools({
  baseDir: './workspace/safe_dir' // Apenas caminhos dentro de safe_dir são permitidos
});
```

3. Proteção Anti-SSRF em `WebTools`

Bloqueia chamadas para endereços de rede privada, `localhost` e metadados de nuvem (`169.254.169.254`, `metadata.google.internal`):

```javascript
const { WebTools } = require('@areumtecnologia/agentic-core');

const webTools = new WebTools();
// URLs privadas ou de metadados disparam SecurityViolation e são bloqueadas automaticamente
```

📊 Observabilidade e Telemetria (v3.3)

Monitore spans de execução, duração em milissegundos e consumo acumulado de tokens com o `TelemetryProvider`:

```javascript
const { TelemetryProvider, AgenticCore } = require('@areumtecnologia/agentic-core');

const telemetry = new TelemetryProvider();

telemetry.on('span:end', (span) => {
  console.log(`[Trace] Span "${span.name}" concluído em ${span.durationMs}ms com status: ${span.status}`);
});

telemetry.on('metrics:tokens', ({ added, cumulative }) => {
  console.log(`[Tokens] +${added.total} tokens (Acumulado:
```

```
${cumulative.totalTokens})`);
});

const core = new AgenticCore({
  agent,
  provider,
  telemetry // Registra automaticamente spans de mensagens, geração de modelo e
execução de ferramentas
});
```

🌐 Orquestração de Tarefas em Grafo (DAG)

Orquestre grafos de tarefas com dependências, condições de branch e
transformação dinâmica de dados:

```javascript
const { TaskGraph, ParallelExecutor } =
require('@areumtecnologia/agentic-core');

const graph = new TaskGraph();

// Tarefa base
graph.add({
  id: 'analise_credito',
  fn: async () => ({ score: 850, aprovado: true })
});

// Tarefa condicional (só executa se aprovado)
graph.add({
  id: 'gerar_contrato',
  deps: ['analise_credito'],
  condition: (ctx) => ctx.results.analise_credito.aprovado === true,
  transformContext: (ctx) => ({ scoreCliente: ctx.results.analise_credito.score
}),
  fn: async (transformedContext) => ({ contratoId: 'CTR-9988', dados:
transformedContext })
});

// Tarefa de rejeição (será ignorada/skipped)
graph.add({
  id: 'notificar_recusa',
  deps: ['analise_credito'],
  condition: (ctx) => ctx.results.analise_credito.aprovado === false,
  fn: async () => ({ emailEnviado: true })
});

const executor = new ParallelExecutor(graph);
const results = await executor.run();

console.log('Contrato gerado:', results.gerar_contrato);
console.log('Recusa:', results.notificar_recusa.status); // 'skipped'
```

```

---

## 🤖 SubAgentes com Delegação de Tarefas

O `SubAgent` permite executar tarefas especializadas em segundo plano utilizando a infraestrutura do agente pai:

- Possui **sessão efêmera própria** limpa automaticamente ao finalizar.
- Suporta **ferramentas temporárias** desregistradas automaticamente no bloco `finally`.

```javascript
const { SubAgent, AgentConfig } = require('@areumtecnologia/agentic-core');

const subConfig = new AgentConfig('Pesquisador', 'Áreum', '...', 'Pesquisador', 'Buscar dados', '...');

const researcher = new SubAgent({
  parent: mainAgent,
  config: subConfig,
  name: 'researcher-subagent',
  tools: [/* ferramentas exclusivas do subagente */]
});

const result = await researcher.execute('Pesquisar relatório financeiro do setor');
console.log(result.output);
```

🏢 Plataforma Multi-Tenant e Persistência

`PlatformManager`
Gerencie múltiplos tenants com instâncias de agentes e métricas isoladas:

```javascript
const { PlatformManager } = require('@areumtecnologia/agentic-core');

const platform = new PlatformManager();
platform.registerTenant('tenant-empresa-a');
const tenantAgent = platform.createAgent('tenant-empresa-a', agentConfig, { provider });
```

`SessionStore` & `InMemorySessionStore`
Persista e restaure o estado das sessões:

```javascript
const { InMemorySessionStore } = require('@areumtecnologia/agentic-core');

const store = new InMemorySessionStore();
```
```

```
await store.save(session.id, session.toJSON());
```

🎯 Eventos do Agente

Escute eventos através de `agent.on(AgentEvents.EVENT_NAME, callback)`:

```javascript
const { AgentEvents } = require('@areumtecnologia/agentic-core');

agent
  .on(AgentEvents.TOKEN, ({ token }) => process.stdout.write(token))
  .on(AgentEvents.REASONING_CHUNK, ({ thought }) => console.log('Pensando:',
thought))
  .on(AgentEvents.TOOL_STREAM_START, ({ toolCall }) => console.log('Iniciando
tool:', toolCall.name))
  .on(AgentEvents.TOOL_STREAM_END, ({ toolResult }) => console.log('Tool
finalizada:', toolResult.name))
  .on(AgentEvents.SESSION_CREATED, ({ session }) => console.log(`Sessão criada:
${session.id}`))
  .on(AgentEvents.SESSION_UPDATED, ({ session, reason }) => console.log(`Sessão
atualizada: ${session.id} (${reason})`))
  .on(AgentEvents.SESSION_EXPIRED, ({ session }) => console.log(`Sessão
expirada: ${session.id}`))
  .on(AgentEvents.SESSION_CLEARED, ({ session, reason }) => console.log(`Sessão
limpa: ${session.id}`))
  .on(AgentEvents.TURN_START, ({ depth, session }) => console.log(`Turno
${depth} iniciado`))
  .on(AgentEvents.RESPONSE, ({ response, reasoning, usageMetadata }) =>
console.log('Resposta:', response))
  .on(AgentEvents.TOOL_CALL, ({ name, args }) => console.log(`Tool chamada:
${name}`, args))
  .on(AgentEvents.TOOL_RESULT, ({ name, result }) => console.log(`Tool
resultado: ${name}`, result))
  .on(AgentEvents.PROVIDER_FALLBACK, ({ failedProvider, nextProvider, error })
=> console.warn(`Fallback: ${failedProvider} -> ${nextProvider}`))
  .on(AgentEvents.ERROR, ({ error, source }) => console.error(`Erro
[${source}]:`, error.message));
```

🔌 Suporte ao Protocolo MCP (Model Context Protocol)

O `McpManager` e o `McpServer` oferecem suporte nativo ao protocolo MCP
(JSON-RPC 2.0 via Stdio):

```javascript
// MCP Client (Importando ferramentas externas)
const { McpManager } = require('@areumtecnologia/agentic-core');
const mcp = new McpManager(agent);
await mcp.registerServer('db', { command: 'node', args: ['mcp-server.js'] });
```

```
// MCP Server (Expondo o agente como servidor MCP)
const { McpServer } = require('@areumtecnologia/agentic-core');
const server = new McpServer(agent);
server.start();
```

🗀 Estrutura do Projeto

```
agentic-core/
├── src/
│   ├── index.js                          # Entry point com exportações
completas
│   ├── AgenticCore.js                    # Core do agente e agentic loop
│   ├── AgentConfig.js                    # Builder de configuração do agente
│   ├── AgentSession.js                   # Estado e histórico da sessão
│   ├── AgentEvents.js                    # Fonte única de verdade de eventos
│   ├── AgentManager.js                   # Gerenciador de múltiplos agentes
│   ├── SubAgent.js                       # Agente especializado efêmero
│   ├── types.js                          # Tipos neutros (Type, ThinkingLevel)
│   ├── utils.js                          # withRetry com backoff e jitter
│   ├── streaming/                        # Streaming em tempo real
(TokenStream)
│   ├── telemetry/                        # Observabilidade e tracing
(TelemetryProvider)
│   ├── tools/                            # Sistema de ferramentas embutidas
│   │   ├── index.js                      # Exportações do sistema de
ferramentas
│   │   ├── ToolRegistry.js               # Registro e descoberta unificada de
ferramentas
│   │   ├── ToolComposer.js               # Composição e chaining de ferramentas
│   │   ├── ToolValidator.js              # Validação de schema e segurança
│   │   ├── ToolMarketplace.js            # Marketplace e descoberta
│   │   ├── ToolSecurityPolicy.js         # Políticas de segurança e rate
limiting
│   │   └── categories/                   # Categorias de ferramentas
│   │       ├── reasoning.js              # Sequential thinking
(record_thought_step)
│   │       ├── web.js                    # Ferramentas web com busca DuckDuckGo
e anti-SSRF
│   │       ├── file.js                   # Ferramentas de arquivo com path
confinement
│   │       ├── database.js               # Ferramentas de banco de dados
│   │       ├── system.js                 # Ferramentas de sistema (com
sanitização e vm sandbox)
│   │       └── communication.js          # Ferramentas de comunicação
│   ├── mcp/                              # Protocolo MCP (Client, Server,
Manager)
│   ├── memory/                           # Memória (Working, Compactor,
Episodic, Semantic, ContextAugmenter)
│   ├── orchestrator/                     # Orquestração (TaskGraph, TaskNode,
```

```
ParallelExecutor, Orchestrator)
│   ├── persistence/                    # Persistência (SessionStore,
InMemorySessionStore)
│   ├── platform/                       # Multi-tenancy (PlatformManager)
│   └── providers/                      # Providers (Google, OpenAI,
Anthropic, Ollama, Nvidia)
├── tests/
│   ├── test_ten_each.js                # Bateria de 30 testes de ferramentas
nativas (100% de aprovação)
│   ├── test_evolution_v33.js           # Suíte completa de testes da v3.3.0
(Streaming, RAG, Segurança, DAG, Telemetria)
│   ├── test_code_review_fixes.js       # Validação de correções da revisão de
código
│   ├── test_v32_improvements.js        # Validação das melhorias da v3.2.0
│   ├── test_bugs_fix.js                # Suíte de teste para correções de
bugs
│   ├── test_retry_classification.js    # Validação de erros retentáveis vs
permanentes
│   └── ...                             # Outros testes de serviço e
integração
├── package.json
└── README.md
```

🧪 Testes

```bash
# Executar suíte padrão completa
npm test

# Executar bateria de 30 testes das ferramentas nativas v3.3.1
node tests/test_ten_each.js

# Executar suíte de novidades da versão 3.3.0
npm run test:evolution

# Executar testes específicos de regressão e correções
node tests/test_code_review_fixes.js
node tests/test_v32_improvements.js
node tests/test_bugs_fix.js
node tests/test_retrocompatibility.js
node tests/test_retry_classification.js
```

📄 Licença

ISC

👤 Autor

Áreum Tecnologia — Software and AI Development Team